

Modeling with Visual Paradigm

Software Architecture 2025–2026

1 Why this lab session?

Why do you even have to go through this tutorial? It is not that hard to make UML diagrams in Visual Paradigm, right?

Well, in this course, you will be working with a large and complex system at different levels of abstraction. You will end up with multiple diagrams of different types that describe one and the same system. These diagrams will have to be consistent.

If you try to maintain consistency by hand, you will fail and quickly become frustrated; guaranteed.¹ The goal of this lab session is to help you understand why Visual Paradigm behaves in the way it does, and how you can benefit from it without becoming frustrated.

Furthermore, knowing how to model well is an important skill in your career as a software engineer. Insights into the notion of modeling gained here are independent of the specific tool.²

On top of that, this lab session will make you familiar with some essential modeling concepts and notations that will be used throughout the course. Putting some effort into creating a decent model will also save you a ton of work for the course (especially near the deadlines), because a dedicated plugin (SAPlugin, section 5) can then automatically detect (many) inconsistencies, and generate a large part of your project report.³ If you do not take care of your model, though, these features will cause you more problems than they will solve.

This text contains a combination of explanations and small assignments (indicated by ‘Try it!’) that you should perform using Visual Paradigm in order to apply and experience for yourself what has been explained.

2 Practical setup

In the pc labs, you can start Visual Paradigm as follows:

```
/localhost/packages/visual_paradigm/current/Visual_Paradigm &
```

If you want to use Visual Paradigm on your own machine, you can download the latest version we have a license for at <https://ap.visual-paradigm.com/kuleuven>. Make sure to update your installation.

2.1 License key

When asked for the license type, choose Academic License. You will have to enter the activation code,⁴ your name, and your student e-mail address to receive a confirmation code for the activation.

¹ From past experiences, you may think that Visual Paradigm sometimes actively works against you to mess up your diagrams, but it actually does some sensible things to help you to achieve consistency across diagrams (really!).

² The notions of models and diagrams are valid in a wide range of modeling tools: Visual Paradigm, MagicDraw, Enterprise Architect, Eclipse Modeling Framework, etc.

³ Without a model, you would have to manually create the entire element catalog of your project.

⁴ See <https://ap.visual-paradigm.com/kuleuven>

2.2 SAPlugin

We have created a Visual Paradigm plugin (SAPlugin) that will support you during this course. Concretely, it will help you to detect inconsistencies against the modeling style we follow in this course, and generate a large part of your project report from the model that you have created. You can find the plugin on Toledo; follow the instructions there to install it. We will use this plugin in this lab, so **install it now**.

2.3 Working together

You can work together with other people on a shared project using VP Online (previously called VPository)⁵. The model is then stored online, and Visual Paradigm offers integrated version control. If you want to use this later (*it is not necessary for this tutorial!*), follow the wizards in Visual Paradigm (in the Team tab) to create an account and set this up.

⁵ Note that this is different from the online editor offered by visual paradigm (called Visual Paradigm Online) — that online editor does not have all the features required for this course, and should thus not be used.

3 Modeling with Visual Paradigm

To model with Visual Paradigm,⁶ you need to know the difference between a **diagram** and a **model**.

⁶ Or any other UML modeling tool, for that matter.

3.1 OK, so what's the difference between a diagram and a model?

There is a big difference between drawing a diagram and creating a model. Drawing a diagram produces a picture; that's it.⁷ It does not matter much how the picture got there; perhaps you needed to use some clever hacks, layering, and styling to create the final picture, but at least it looks exactly the way you wanted ('pixel-perfect').

⁷ It can even be a photo of a white-board drawing.

We will call a graphical element on a diagram a **view** in this tutorial, just like Visual Paradigm does.⁸ Examples of views are the well-known rectangle for a UML class, the stick man for an actor in a use case diagram, or the lifelines for runtime instances in a sequence diagram. Some typical attributes related to views and diagrams are the coordinate and size of the rectangle on the diagram, and the color in which it is drawn.⁹

⁸ This concept of a view is similar to, but slightly different from, the notion of a view and viewpoints we will encounter in the course.

⁹ In Visual Paradigm, that is an annoying blue color by default, which we will get rid of shortly.

The drawings (or, sometimes, the code snippets) make use of the **concrete syntax** or **notation** of the modeling language. It is what you can immediately see, but it is not necessarily a complete or accurate description of the underlying model.

A **model**, then, is a consistent, structured collection of (**model**) **elements**, that are related to each other in a meaningful way. UML¹⁰ is a modeling language, not a drawing language. Its visual aspects (namely UML diagrams) happen to be the most well-known part of it, but UML is much more than that. For example, a Class is a model element in UML, which is related to other model elements such as

¹⁰ Which stands for "Universal Modeling Language".

the Attributes of the class, the Operations offered by the class, or the Class(es) from which it inherits. The UML specification contains much more model elements than diagram elements. Hence, there are things in a UML model that you cannot show graphically. The UML specification defines the structure of and relationships between these model elements, which is called the **abstract syntax** or **meta-model** of the modeling language.

On top of that, each model element has a certain meaning: what does the model element intend to convey? How should you interpret it? How does it behave (by itself, or in relationship to other elements)? This meaning is called the **semantics** of the modeling language. The semantics can be specified informally¹¹, or more formally/mathematically.¹²

The most important thing to keep in mind in order to efficiently use a modeling tool such as Visual Paradigm is that **a single model element** (think: a component) can be represented by **multiple views** (think: rectangles) on **multiple diagrams** (think: pictures).

¹¹ As in the UML specification

¹² E.g., fUML <https://www.omg.org/spec/FUML>

3.2 Working with models in Visual Paradigm

Time for some action!

Try it! 1. Fire up your Visual Paradigm and change the default color of shapes and connectors to white *now*, via Window ▸ Project Options... ▸ Diagramming ▸ Shape (respectively Connector) ▸ Shape fill format (respectively Background).

Try it! 2. Create a new Component Diagram (Diagram ▸ New ▸ Component Diagram). On that diagram, create a new component called 'Gateway'.

On the left, you should find the **Model Explorer** in one of the tabs (if not, go to View ▸ Panes ▸ Model Explorer). This is your primary way for accessing the model that underlies all the diagrams in the project. The model only contains the 'Gateway' component for now. Double-clicking on the 'Gateway' model element in the explorer opens a dialog ('Component Specification') in which you can specify more properties for the element than you will ever need. Not all of these properties (in fact, only a minority) are reflected back into the diagram.

Copying an element We need another component 'OnlineService'. Lazy as we are, let us copy and paste the Gateway component.

Try it! 3. Select the Gateway component on the diagram and copy it (using the menu, or your favorite shortcut equivalent to Ctrl-C). Don't paste it anywhere yet!

For pasting, Visual Paradigm offers two options (you can find them at Diagram ▸ Paste):

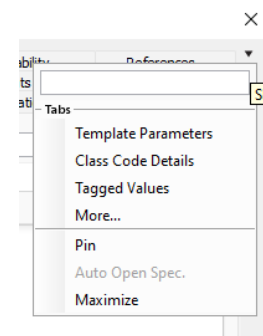


Figure 1: Are you missing a property tab in the specification dialog? Visual Paradigm hides some less frequently used tabs. Click the small triangle in the upper right corner to show additional tabs.

1. **Paste view** (the default, Ctrl-V). This creates a second view, linked to the same Gateway component in the model.

Try it! 4. *Paste the view. See how the Model Explorer didn't change? You've just copied the view (the rectangle), not the model element. Leave the second view where it is for now.*

But we still do not have an OnlineService component, so let us try the second option.

2. **Paste model element.** This creates a new model element, together with a view. Watch the Model Explorer while doing the following.

Try it! 5. *Paste the Gateway again, but now as a model element (using the menu). The new model element is called 'Gateway2'; rename it to 'OnlineService'.¹³*

Notice how OnlineService now sits next to Gateway in the Model Explorer.

Perhaps creating a new OnlineService component would have been more efficient than copying after all...¹⁴

Master view and auxiliary views For every element, there will be one view designated as the element's '**Master view**'. Typically, the master view is the view you have used to do create the element the first time, but that can be changed (see below). Additional views are referred to as '**Auxiliary views**'. You can see the type by the 'M' and 'a' letter in the top left of the view when you select it (fig. 2).¹⁵

The different views of a model element all refer to that same (model) element. The 'rename' command works on the model element, thus all views are automatically kept in sync (i.e., renaming affects the name on all views). They may have different visualizations, though. One view of the Gateway component can be colored in blue, while another is green. Also, some views may hide some of the attributes of the underlying element (e.g., the operations of an interface, properties of a class).

Try it! 6. *Create a second component diagram (Diagram ▸ New), and copy/paste the view of the Gateway component onto it. Rename the Gateway component to GW, and check that it is renamed on all diagrams. Now rename it back to Gateway. Check that changing its color (which you can do using the Property pane: View ▸ Panes ▸ Property) only applies to that particular view.*

The master view can be used to manipulate the location of the element in the model hierarchy. At this moment, the Gateway component is located under the root of the model. Moving the master view to some other container element (e.g., another component, a package, or a node) moves the element within the model structure as well. Doing the same change with an auxiliary view does not have this effect.

¹³ Note that Visual Paradigm is not always very smart about numbering. If you copy and paste the Gateway2 component, you will get Gateway22, not Gateway3. Nevertheless, names with a numeric suffix should be avoided anyway because they are confusing.

¹⁴ Besides copy and paste, Visual Paradigm also supports a Duplicate command (Ctrl-E). This does the same as copy+paste model element, but saves you some clicks/key presses.

¹⁵ When the master view is the only view for the element, the letter M does not appear.

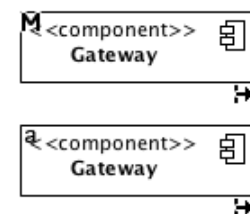


Figure 2: Master view (M) and auxiliary view (a) of the same Gateway component.

Try it! 7. Enlarge the rectangle of the *OnlineService* component such that it has some whitespace. Move the *Gateway* master view in this whitespace, and see what happens in the Model Explorer. Move the *Gateway* back outside, and try the same with the auxiliary view of the *Gateway*. What is the difference?

You can also create an element directly in the Model Explorer (right click, Model Element ▸ New Model Element. . .). In that case, there is no view for the element. The element's master view is then set to the Model Explorer itself. You can change the master view of an element by right-clicking it in the Model Explorer and selecting Show View. . . ▸ Set Master View. . .

Creating a new auxiliary view You can create a new auxiliary view by copy/pasting (as above), but there are also three other ways.

First, you can drag the element from the Model Explorer onto your diagram, which will create a new auxiliary view for it. If the element is related to elements already on the diagram (e.g., you add an interface that is provided by one of the components on the diagram), the links will be shown as well.

Alternatively, you can create a new element, and give it the name of an existing element (of the same type). If both elements have the same parent, Visual Paradigm will ask you whether you really want to create a new view for that element.¹⁶

Keep in mind that Visual Paradigm requires elements with the **same parent** and of the **same type** to have **unique names**. This is the reason why Visual Paradigm suggests to create an auxiliary view, rather than creating a new element with the same name. In this course, we will go a bit further, and impose the following:

¹⁶ **Caution** though: when you do this, the original new element (before you gave it the existing name) remains in your model (without any views), and you will have to delete it manually.

■ **Convention 1:** *All components, interfaces and data types should have a globally unique name.*

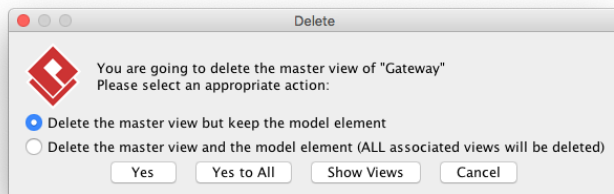
Hence, the name of an element uniquely defines it. This convention significantly reduces confusion when interpreting diagrams.

Finally, once you relate elements to each other, you can also add auxiliary views for other existing related model elements on the diagram. Right-click on the element, and pick Related Elements ▸ Visualize related model element. In the dialog that appears, you can pick which elements to add to the diagram. Because we haven't added any relations between our elements yet, this dialog is still empty, but that will change in section 4.2.

Try it! 8. Try to create a second component that also has the name *Gateway*.

Deleting a view **Deleting an auxiliary view** always removes just the view, and never the element itself.

When **deleting a master view**, the situation becomes a bit more complex. If the master view is the only view for that element, Visual Paradigm asks you¹⁷ ‘X has no referenced view. Delete X from the repository?’. The repository here refers to your model. Hence, you are given a choice between removing the element view from the diagram (but keeping the element itself), or removing the element and its view from the model entirely.



¹⁷ If it doesn't ask you anything, you may have disabled the warning prompts at some earlier time. You can re-enable them in Window ▸ Application options ▸ General ▸ Project, and set 'Delete model element when deleting its view' to 'Ask' and tick 'Confirm shape deletion'.

Figure 3: This dialog appears when you try to delete the master view of an element. The two options have very different effects.

If there are other views besides the master view you are deleting, Visual Paradigm asks you (fig. 3) to choose between deleting just the (master) view and keeping the element, or deleting the element and all associated views (on all diagrams). In the first case, you just delete the master view; all auxiliary views remain in place, and the element in the model explorer becomes the new master view. In the second case, all traces of the element in your project disappear. Evidently, you should think carefully about what you want.

Try it! 9. Delete the master Gateway view, without deleting the auxiliary views. Then delete the Gateway element and all its views at the same time.

4 Modeling software architectures in UML

Now that we know the difference between model elements and views, and you have been convinced of the importance of a model, and know how to make Visual Paradigm do what you intend, we can turn to modeling a software architecture.¹⁸

¹⁸ To be clear: the way in which we present modeling in UML here is not the 'only right way'; it is just the set of conventions that we will use throughout the course (supported by the SAPPlugin) to make your life easier.

■ **Convention 2:** The following diagram types are used for the purposes described in the table below:

<i>We model (Architectural viewpoint)</i>	<i>UML Diagram type</i>
<i>Runtime entities and their interactions (Component & Connector — Client-Server)</i>	<i>Component diagram</i>
<i>Design structure (Module — Decomposition)</i>	<i>Component diagram</i>
<i>Mapping to physical infrastructure (Allocation — Deployment)</i>	<i>Deployment diagram</i>
<i>Detailed behavior/interactions (Component & Connector — Process)</i>	<i>Sequence diagram</i>
<i>Details of data types and interfaces (Module — Class)</i>	<i>Class diagram</i>

4.1 Data types

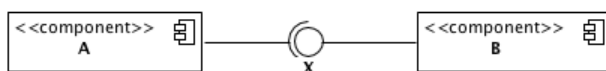
Data types are best modeled as classes on a **Class diagram**. You can enter attributes and their type more quickly using the syntax `attributeName: Type`. Note that a data type is only used as a **container for data**. It has no functionality (methods); all functionality must be documented in an interface (see next subsection).

Try it! 10. Create a new empty project, and add a new Class Diagram. On that diagram, create a data type (class) *SensorReading* with 3 attributes *value* (int), *sensorID* (string) and *timestamp* (long).

4.2 Components and connectors (client-server view)

Components and connector are modeled on a **Component Diagram**. A component in UML is represented by the «component» stereotype, whereas to model a connector, we adopt the notation of matching provided interfaces with required interfaces.

To model a provided interface of a component (i.e., the functionalities it can/will process in the context of the interaction), you need to create an **Interface Realization** relationship between them. Perhaps somewhat counter-intuitively, you create this relationship in Visual Paradigm *from* the interface *to* the component (think: ‘is realized by’). Click on the interface and drag the connection symbol  that appears to the component. When you release the connection, click on the Realization (the arrow with the R). The result is a component with a ‘ball’ (or ‘lollipop’) attached using a solid line (Component B in fig. 4 provides interface X).¹⁹



To model that a component **requires** a certain interface (i.e., it uses or depends upon the operations defined in the interface), you need to create an **Interface Usage** relationship. In Visual Paradigm, this goes *from* the component *to* the interface (i.e., in the opposite direction as the realization). Click the component and drag the connection symbol  that appears to the interface. When you release the connection, click the Usage (the arrow with the U). The result is a component with a ‘socket’ (or ‘hand’) attached (Component A in fig. 4 requires interface X).²⁰

Note that Visual Paradigm also displays an Association between an interface and a component as a required interface. Although you will not see this distinction in the graphical representation, this is **semantically incorrect** (at least according to our conventions), and the SAPplugin will eventually complain about it.

¹⁹ Alternatively, you can create the relationship from the toolbox on the left. Realization can be found as a special case of the Generalization arrow.

Figure 4: Component A requires interface X; component B provides it.

²⁰ Alternatively, you can again create the relationship from the toolbox on the left.

■ **Convention 3:** Only use Realizations or Usages to connect interfaces and components.

Realization and Usage relationships are also model elements, but are not shown by default in the Model Explorer. You can show or hide them by right-clicking some empty space in the Model Explorer, and (de-)selecting ‘Show Relationships’.

A final note: specifying an interface’s operations is often easier on a Class diagram. At this point in the tutorial, you should understand that you can indeed create an (auxiliary) view of the interface on a Class diagram. There, you can easily add the operations. You can do that quickly using the following syntax:

```
operationName(param1: Param1Type, param2: Param2Type): Return Type
```

Visual Paradigm will parse what you type and create links to the data types (if they exist, of course). Exceptions for operations can be added through *Raised Exceptions* tab on the operation’s specification window.

Try it! 11. *Create a new Component Diagram. Add a Gateway component, an OnlineService component and Sending interface. The Gateway must require the Sending interface, and OnlineService provides it. Create a class diagram, add an auxiliary view for the Sending interface, and give it one operation: `send(gatewayID:string, reading:SensorReading):boolean`. Open the specification dialog of your new operation (select it and press Enter), open the Parameters tab, and double-click the reading parameter. Ensure that the type of the parameter indeed refers to the SensorReading class you’ve created earlier by checking the dropdown box.*

4.3 Decomposition into components

To flesh out some more details of a specific component, you can further decompose it. This implies that the parent component is no longer interpreted as a single *process on a server*,²¹ but it has become a logical grouping of processes belonging together.²²

For the decomposition, you can start from a new component diagram. However, Visual Paradigm has special support for attaching diagrams to an element. If you right-click an element, you can pick Sub Diagrams ▸ Add New Diagram... The diagram then becomes linked to the element, and you can quickly jump to it using the arrow at the bottom right of the component rectangle.

Once the new diagram is opened, first add the component that you are decomposing and enlarge it. You can then add sub-components inside that component (in the white space).

Try it! 12. *Create a decomposition of the Gateway component by creating a new subdiagram of type ‘Component diagram’. Start by adding an auxiliary view of your existing Gateway component, enlarge it, and add two sub-components: Collector and Transmitter. Collector pushes data to Transmitter through a Transmit interface that you should also create now. The Transmit interface has one operation: `transmit(data: SensorReading): void`.*

²¹ Element with run time presence, like a thread or a service

²² For example, to ease their deployment together.

■ **Convention 4:** *For this course, we will require that, if a parent component requires or provides an interface, then at least one of its sub-components also requires or provides that interface.*

This convention means that there is no ‘residual’ functionality in the parent component; it is fully implemented by the composition of its children. In our example, the Transmitter is the sub-component that uses the Sending interface which we previously designated as an interface required by the Gateway (parent) component. The SAPLugin will check this consistency requirement, and warn you in case it detects a problem.

Try it! 13. Add the sending interface to the outside of the Gateway component. Visual Paradigm will automatically show the Usage relation between the Gateway and the Sending interface. Now also add a usage relationship between the Transmitter sub-component and the Sending interface.

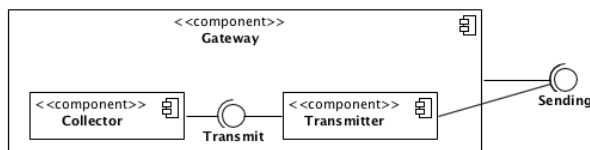


Figure 5: Both parent and child component must require the interface.

The model is now semantically correct, but poses a graphical challenge: there are two lines (representing Interface Usage relationships) to the Sending interface (one from Gateway, one from Transmitter), as shown in fig. 5. For convenience, we would only like to see the line from Transmitter to Sending. By placing the Sending interface outside of the Gateway component, the visualization indeed already implies that it is also required by Gateway.

Try it! 14. Delete the usage line (but *not* the model element!) from Gateway to Sending. You may have to re-route one of the lines to see and select the right one. Verify that the Gateway component still requires the Sending interface (although it is no longer shown explicitly on the diagram). You can see this in the ‘Relations’ tab of the Gateway’s specification dialog²³.

²³ You may need to click the small black triangle at the right to make the Relations tab visible.

4.4 Decomposition into modules

In this course, module decompositions of components are also modeled on a **Component Diagram**.²⁴

Attention! Client-server views and decomposition views have a very different purpose. Whether you put a specific UML component in the client-server view or in the decomposition view will have an impact on how these diagrams are read and interpreted! A client-server view documents run-time interactions and communication between components,²⁵ whereas a decomposition view documents modules as units of design or implementation.

To avoid confusion and to tell Visual Paradigm whether you are specifying a component or a module, we adopt a stereotype notation. If you decompose a component and want to specify design structure, add the stereotype «module» to the sub-components.²⁶

To create a decomposition, create a component diagrams similar

²⁴ Other alternatives are possible, such as a Composite Structure Diagram or a plain Class Diagram, but we will not use them in the context of the course.

²⁵ Think processes running on a server.

²⁶ This is also how the SAPLugin makes the distinction between client-server views and decomposition views. It checks whether there are components with the «module» stereotype on the diagram.

as before.²⁷ Once the new diagram is opened, add the component that you are decomposing and enlarge it. You can then add the modules inside that component (in the white space).

²⁷ You can again pick between a new component diagram or a sub diagram of the component you are decomposing into modules.

When you decompose a component into modules, you have effectively established that the parent component is a single “*process on a server*” and started carving out the decomposition view to show the inner design structure of that component. Once you have created a decomposition into modules, you cannot decompose that same component into other components. You cannot decompose a module into components (that makes no sense), but you could further decompose modules hierarchically into other modules, theoretically as often as you would like. Do keep in mind that each layer of further hierarchical decomposition and refinement will generate extra complexity for the reader of your model to understand what is going on, and for you to document. In most cases, decompositions three levels deep will be more than sufficient.²⁸

²⁸ Level 1: the entire system as depicted in the context diagrams, level 2: components in the client-server view, and level 3: modules in the decomposition view.

Try it! 15. *To create the Transmitter, we will need a module to marshal objects into messages, a `MarshallLibrary` and a module to establish and control connections, a `ConnectionManager`. These libraries package routines and functionality that will be used in the creation and establishment of a Transmitter process, but they will not be separate processes (components) – instead they are modules. Decompose the Transmitter and specify that it will consist of these modules. Don’t forget to stereotype modules as such (use the right-hand mouse button to find Stereotypes, or add them in the component specification dialog).*

You will now have a component with some modules. However, since they are not connected, it is unclear how the functionality of the Transmitter is realized, and the SAPlugin will complain. We will need to specify the relationship between these modules and the interfaces of the Transmitter.

■ **Convention 5:** *For this course, we will require that, if a parent component requires or provides an interface, then at least one of its modules (if it has any) also requires or provides that interface.*

Try it! 16. *Model that the `ConnectionManager` implements the `Transmit` interface and requires the `Sending` interface. Introduce a new `Marshal` interface. This interface is provided by the `MarshallLibrary` and required by the `ConnectionManager`. Also specify a marshalling operation.*

Modeling external libraries in your decomposition views. You may now notice a problem, if you consider this `MarshallLibrary` to be an external library that you would want to reuse in multiple decompositions. Indeed, your module can only have a single parent in your model.²⁹ If you would want to reuse your module in other decompositions, the SAPlugin would complain that the other parent components also need to require the `Marshal` interface to keep the interfaces consistent (as required by Conventions 4 and 5).

²⁹ In this case, the component you decomposed.

Try it! 17. To fix this problem, use the Model Explorer to drag your external library module to the root of the model tree.³⁰

Now, because your library module no longer has the **Transmitter** component as parent, the SAPPlugin will assume it is an external library and will no longer complain about its interfaces.³¹

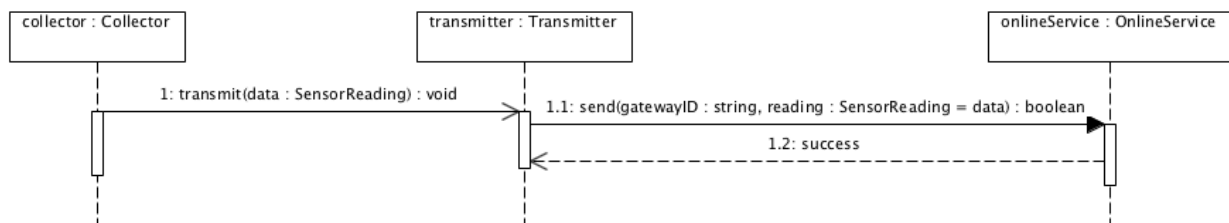
4.5 Interactions

Interactions between components can be modeled on a **Sequence Diagram**. The lifelines on a sequence diagram represent component instances, and the arrows represent operations that are called. The easiest way to create a lifeline for a component is to drag that component from the Model Explorer onto the sequence diagram.

To make one component call an operation of the other component, add a **Call Message** between them.³² Do not type any text when creating the message; you can select an operation by right-clicking **▷ Select Operation ▷ Select Operation...** Visual Paradigm then shows you a list of operations that are provided by the target component,³³ from which you can choose one. The call message is now linked to the operation; if its name or signature changes, the sequence diagram is updated automatically.

■ **Convention 6:** It only makes sense to call an operation if the **source component requires the interface through which the operation is provided**.

Visual Paradigm does not take this convention into account, but the SAPPlugin will complain if you violate it.



Try it! 18. Create a new Sequence Diagram. Add three lifelines; one for an instance of the **Collector** component, one for the **Transmitter** component, and one for the **OnlineService** component. Model the **Collector** calling the **transmit** operation on **Transmitter**, which then calls the **send** operation on the **OnlineService**.

Synchronous vs asynchronous messages When calling operations, there is an important difference between synchronous and asynchronous messages that you should pay attention to.

When using a **synchronous** message,³⁴ the caller **blocks** until the result (or an error) has been returned. This makes it possible

³⁰ Actually, your module can be anywhere, as long as it does not have a component or a module as parent.

³¹ **Caution!** You may think to apply this *trick* for all your modules. But, what you model in such a case is that you have no decompositions at all and that all modules in your project are external. Not only is this incorrect, you will also not be alerted to interface inconsistencies in your decompositions.

³² In the Visual Paradigm toolbox, call messages can be found by expanding regular messages.

³³ I.e., the operations from the interfaces provided by the target component. Visual Paradigm does not consider the required interfaces of the source component.

Figure 6: A sequence diagram where the argument value of **reading** in the **send** call is linked to the **data** parameter from the **transmit** call.

³⁴ They are the default in Visual Paradigm.

to receive a return value, which you model using a **reply message** (shown as a dashed arrow). Synchronous messages are shown using a solid arrowhead on a sequence diagram.

An **asynchronous** message, on the other hand, does not block the caller. It is sent and then forgotten. Hence, the return type of the called method should be void, and drawing reply messages (dashed arrows) makes no sense for asynchronous messages. If you do need to process the result from sending an asynchronous message, you will have to foresee a callback mechanism of some sort (and link the reply to the request). Asynchronous messages are shown using an open arrowhead on a sequence diagram.³⁵

³⁵ On top of that, it may be helpful to explicitly add an 'Async' suffix to asynchronous operation names to make this intention clear.

Try it! 19. *Think about which of the calls in your sequence diagram could be asynchronous (hint: look at the method signature), and model them as such (by ticking 'Asynchronous' in the message's specification dialog).*

Parameters When using Call messages on a sequence diagram, Visual Paradigm by default displays the parameter names and types as they appear in the definition of the called operation. Sometimes, you may want to model that the parameter or result of one call is used as an argument in the next. Relying just on the correspondence of the parameter names is a sure path to errors and ambiguity: it introduces an implicit constraint that the parameter names of different operations must be equal, which is violated as soon as one of the parameters is renamed.

Fortunately, there is a way to model this more explicitly, using the properties dialog of the Call Message, tab Arguments. The argument values are determined by position, and you can add argument values by adding a text or model element. Note that an empty text element is not displayed, and can thus be used to skip parameters for which you don't want to provide a value.

Adding a text element with value '*text*' just displays "*= text*" after the parameter. Instead, you can also make the value refer to a model element (e.g., the parameter of an operation). Visually, that will also add "*= name*", just like the text element. In this case, however, renaming the parameter also renames the argument value.

Try it! 20. *Make sure that the reading parameter of the send call on your sequence diagram refers to the data parameter that was received from the Collector (fig. 6). The data parameter is a model element that resides under the transmit method of the Transmit interface. Rename the data parameter to something else, and verify that the sequence diagram reflects this in both places.*

Components vs. modules When creating sequence diagrams, you can typically create lifelines for two types of elements: components or modules. Given that these represent either runtime information or design structure, these are typically not combined.

■ **Convention 7:** *For this course, we will require that, a sequence diagram contains either components or modules for the lifelines, but not both at the same time.*

This means that either (i) you document runtime interactions and, thus, all your lifelines represent components, or (ii) you document the internal design structure of a component and, thus, you only use that component's modules for the lifelines.

When documenting the interactions in the internal design structure of a component, you may want to illustrate how those internal interactions are the consequence of an incoming operation.³⁶ For that, you can create a lifeline for the external interfaces of the parent component to visualize how your modules interact with their environment through the external interfaces.

³⁶ Or how they make outgoing calls to external operations.

4.6 Deployment

Physical distribution of components over machines can be modeled using a **Deployment Diagram**. Machines are represented by Nodes, which are shown as 3D boxes.

■ **Convention 8:** *To deploy a component on a node, we will just place the component within the node.*³⁷

Take care that, when deploying a component, you do not want the node to become the parent of the component *in the model*.³⁸ Moving the master view onto a node may lead to this situation (cfr. section 3.2). Again, the SAPPlugin will warn you if you violate this convention. Also, a component needs to be fully inside the node. You can check that it is deployed correctly by moving the node; the component should move together with the node.

³⁷ Technically, this is not valid UML, and you would be required to create artifacts that manifest certain components, and deploy those artifacts, but we will not be bothered with that.

³⁸ A single component could be deployed on many nodes (e.g., for replication).

Associations between nodes represent communication channels. Two nodes can see/reach each other only if they have an association between them. You can think of this association as a network connection, but it represents a logical link more than a physical one. Just like associations on a regular Class diagram (which you should already know from earlier courses), associations on a deployment diagram can have multiplicities on both ends.

To ensure consistency between the component and deployment diagrams, we impose the following.

■ **Convention 9:** *Components that need to communicate with each other should be deployed on the same node, or on separate nodes with a (direct) communication channel between them.*

An example of a deployment diagram is shown in fig. 7. Keep in mind that a multiplicity greater than one implies that there might be concerns related to addressing the right node. For example, when there is more than one node N, you should clearly document

whether a node M should communicate with every N (broadcast), randomly select one (e.g., load distribution using round-robin), talk to a fixed one, etc.

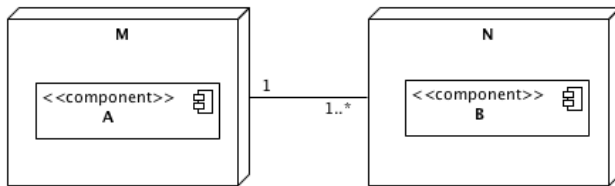


Figure 7: An example of a deployment diagram where component A is deployed on node M, and component B on node N. Each M-node can communicate with many N-nodes (but always at least one); each N-node must communicate with exactly one M-node.

Try it! 21. Create a deployment diagram, and two nodes (ClientDevice and Server). Create an association between the two nodes, and set the multiplicities such that one server can be connected to many ClientDevices, but each ClientDevice connects to a single server. Deploy the Gateway component on the ClientDevice node, and the OnlineService on the Server node.

4.7 Documentation

So far, we have only given names to our elements. While choosing good names is important, it is insufficient to convey the exact meaning on what the purpose of each element is. For that reason, all model elements (and for this course, the components and operations in particular) need to be enriched with documentation.

You can enter the documentation for an element in its specification window, or you can open a dedicated Description popup window (View ▸ Panes ▸ Description) which follows your selection.

Try it! 22. Enter a description for all components and operations.

The SAPlugin will incorporate this documentation into the report that it can generate for you.³⁹

5 SAPlugin

The SAPlugin will help you in this course to create models that follow our conventions by performing a number of consistency checks. It also generates documentation⁴⁰ from your model, which you will have to hand in. The plugin also has a manual, which describes the features in more detail, and is available on Toledo and from within the plugin.

5.1 Consistency checks

As mentioned before, the conventions we have outlined so far, as well as the checks performed by the SAPlugin, are defined specifically for this course to make modeling (and document generation) as easy as possible for you. They sometimes rely on violations against UML, or do not consider all valid variations within UML.

³⁹ And warn you if documentation is missing by highlighting missing entries in red. This means you should always check the output of your report.

⁴⁰ Client-server view, decomposition view, deployment view, process view, and the element catalog (with components, modules, interfaces, operations, nodes, exceptions, and data types).

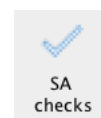


Figure 8: Consistency checks icon

Warning! It is not because the plugin says everything is OK, that you have a good project! It merely means that you adhere to some modeling conventions. The plugin sometimes uses heuristics and does not check everything.

Warning! It is not because the plugin says there are problems, that you have a bad project! Conventions can be ignored if you have a good reason to do so. But: it may indicate weak points.⁴¹

⁴¹ For example, sloppy modeling or missing documentation.

Try it! 23. Press the ‘SA checks’ button of the plug-in. You can pick the checks to perform (all are checked by default). The message pane at the bottom shows the results of the checks (if you don’t see it, open it from View ▸ Panes ▸ Message). If you followed the tutorial correctly, you should not get any warnings.⁴²

⁴² Except on the absence of some diagrams.

The SAPlugin manual provides the complete list of checks that are performed.

Try it! 24. Press the ‘SAPlugin Manual’ button of the plug-in. Your PDF reader should open with the manual of the SAPlugin. Scroll down to section 3 for a complete overview of the available checks.

Try it! 25. Try to modify your current model to make the above consistency checks fail one by one. Note that some are interdependent, and triggering one will also trigger the other. If you don’t want to do them all, we suggest to at least try ‘Interface never provided’, ‘Interface not required by parent’, ‘Call consistency’, and ‘Components cannot communicate’.

5.2 Structuring your project

The SAPlugin offers a button to organize your project. It will create top-level packages for Components, Datatypes, Deployment, Exceptions, Interactions and Interfaces, and move the elements into these packages (see fig. 10).

Try it! 26. Press the ‘SA project structure’ button. You can select which element types are to be structured; by default, all are selected. The elements will be moved into top-level packages depending on their role.

Note: you could do this manually as well, except for the interactions (sequence diagrams). For an unknown reason, they can only be moved programmatically within Visual Paradigm.

5.3 Generating documentation

To help you hand in your architecture at the end of the semester, the SAPlugin contains a button to generate a LaTeX report. Included in that report are all component, deployment, and sequence diagrams (no class diagrams!), an element catalog with all descriptions, and cross-links between the elements.

Try it! 27. Press the ‘Export to LaTeX Report’. Enter a path where to generate the files. Wait patiently until the message “Exporting finished” appears in the message pane (generating the report for a large architecture can take quite a while). Build the report.tex file using pdflatex.



Figure 9: Structure project icon

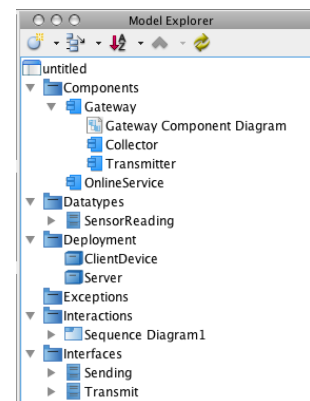


Figure 10: Example of the model structure which the SAPlugin will create.



Figure 11: Export to LaTeX icon

Note that the files `diagrams.tex` and `catalog.tex`, as well as the image files, will be overwritten the next time you generate the report. The `report.tex` and `rationale.tex` files are not generated again if they exist, because you will have to modify them manually.

5.4 Bug reporting

We have done our best to make the SAPlugin as robust and bug-free as possible. Nevertheless, if you discover any situation where the consistency checks fail, encounter weird behavior (of the tool), or can make the plugin crash, we would love to hear about that at SoftwareArchitecture@cs.kuleuven.be!

6 Modeling principles for this course

While modeling in this course, it is good to keep the following principles in mind.

Include everything only once. A single item of your architecture should appear in your model exactly once. You can make multiple views of that model element, and sometimes it may even be desirable to depict a single element twice on the same diagram. But all these views should refer to the *same* underlying model element. In this course, we use the convention that *views with the same name refer to the same model element*, even if Visual Paradigm does not enforce it. Hence, all model elements should have a unique name.

Keep your model clean. Remove elements from your model that are no longer in use. Clutter in your model generates noise and leads to confusion, so make sure to clean up your model elements as well, and not just their views. This is easier if you also follow the next guideline.

Keep your model structured. Try to maintain a clear and logical, hierarchical structure for your model. For example, you can ensure that sub-components are always part of their parent components. Furthermore, you can group all elements of a particular type in a dedicated package (e.g., Components, Interfaces, Nodes, Datatypes, Exceptions, ...). The SAPlugin can help you do this (albeit limited to the conventions it has been programmed for, of course); you can trigger this by pressing the ‘SA project structure’ button.

Use references. If what you are modeling conceptually refers to something else in your model, make that link explicit. For example, a message in a sequence diagram should actually refer to the operation that is being called, rather than just being decorated with text that corresponds to the intended operation name. These explicit references allow changes (e.g., renaming) to be propagated consistently and automatically throughout your model, and they

enable automated consistency checks (e.g., is the corresponding interface required by the calling component).

Consistency over looks. In modeling, the most important feature is that your model is correct and complete. Therefore, do not sacrifice model consistency or completeness just for the sake of improving its visualization. That does not mean that you should neglect what your diagrams looks like, of course. In the end, the diagrams are what other humans will look at and try to comprehend, so they need to be readable. Inconsistencies make understanding a set of diagrams much harder, regardless of whether they are pretty or not. Furthermore, an unreadable figure may indicate that some refactoring in the model is due.

7 *Submission instructions*

Because it is so important to get acquainted with modeling in Visual Paradigm for the remainder of the course, we ask you to submit your vpp file after following the tutorial.

Submission We expect a single vpp file in the submission.

- The submission is individual. Each member of your group submits their own vpp file. If you did part of the tutorial together during the exercise session, you can continue from the same file.
- You do not need to retain or submit intermediate versions of your vpp project.
- You do not need to keep intermediate results in your model, it is perfectly fine to modify the result from a previous '*try it*' to complete the next one.

Grading This submission is *not* graded. Nevertheless, it is really important that you are familiar with these modeling concepts in order to successfully complete part 2 of the course.

Delivery Submit the final vpp after this tutorial in Toledo by the indicated deadline.

Happy modeling!

The Software Architecture Team

A Visual Paradigm Online

This appendix briefly describes how you can create a visual paradigm online workspace so you can work on the same Visual Paradigm project with your teammates.

A.1 Creating a workspace

To create a workspace, **one** member of your team needs to sign up for VP online. Go to Team ▸ Select Repository ▸ Sign up VP Online (fig. 12). Make sure the other team members do not click on “Sign up VP Online”, otherwise you will all have separate workspaces.

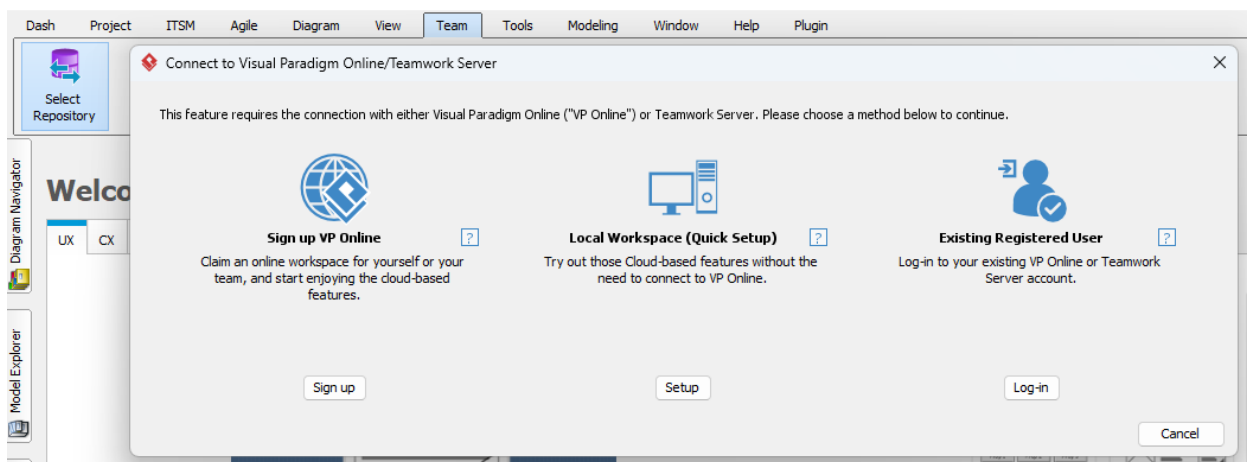


Figure 12: Connect to Visual Paradigm Online Dialog

After signing up, you will receive an email with your workspace ID. The workspace ID is also visible in the url:

`https://online.visual-paradigm.com/w/<workspaceid>/admin.jsp#members`

After creating the workspace, you can invite your other team members as project members. They will receive an email to register an account so they can join the workspace.

A.2 Joining an existing workspace

To join an existing workspace, go to Team ▸ Select Repository ▸ Existing Registered User (fig. 12). Make sure you do not click on “Sign up VP Online”, otherwise you will create a separate workspace instead of joining the existing one.

A.3 Troubleshooting if you did create separate VP Online repositories

Visual Paradigm does not offer an easy way to permanently sign out of a repository.⁴³

To permanently remove the connection to the existing repository you can either create a new workspace, or remove the `teamworkUser.xml` file in your existing workspace.

⁴³ You can logout via Team ▸ Utilities ▸ Logout. But, this will only temporarily log you out. Afterwards, if you click Team ▸ Login, you will automatically return to the original online repository.

To find out where your workspace is, go to Help ▸ Switch Workspace (fig. 13). The directory listed there is your workspace directory.

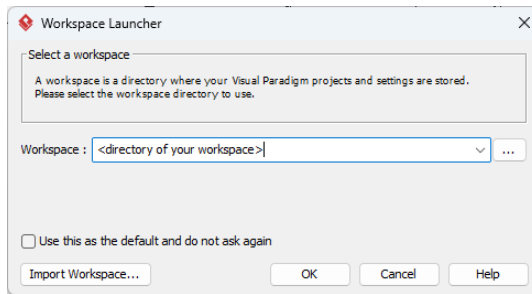


Figure 13: Visual Paradigm workspace

Go to this directory with a file explorer, and delete the file `teamworkUser.xml`.⁴⁴ Alternatively, you can also create a new workspace.

When you delete the file and start Visual Paradigm again, you can go to Team ▸ Select Repository, as before.

⁴⁴ Make sure Visual Paradigm is closed when deleting this file, otherwise it will be recreated!